

Cortex: AI as PID 1

An AI-Native Operating System Kernel Where the
Inference Engine Isn't an App — It's `init`

Elevate Foundry
github.com/elevate-foundry

June 2025

Abstract

Modern AI-assisted computing treats large language models as user-space applications: launched on demand, isolated from the system, and discarded after use. This mirrors the pre-Unix era, where every program implemented its own I/O routines. We propose **Cortex**, an architecture in which the LLM inference engine operates as PID 1 — the always-running, kernel-adjacent process that mediates between user and hardware. Cortex constructs a tiered model hierarchy L0–L7, routes each request to the smallest model capable of handling it via an L0 classifier, and escalates through cross-family challenge and multi-model swarm consensus when confidence is insufficient. We formalize the routing function, define a cross-family confidence metric grounded in independent failure probabilities, and describe the swarm aggregation mechanism. The system runs on any hardware via a GGUF-first format strategy and presents a universal API compatible with OpenAI, Anthropic, and tool-calling clients.

1. Introduction

1.1 The Problem

Let $\mathcal{M}$ be a language model and x a user request. Standard deployment routes every x to a single, typically maximal, model $\mathcal{M}_{\max}$:

$$y = \mathcal{M}_{\max}(x) \quad \forall x \quad (1)$$

This has three structural inefficiencies:

1. **Overprovisioning.** A classification task ($|x| < 50$ tokens) consumes the same compute as a multi-step coding task. The expected compute is $\mathbb{E}[\text{cost}] = c(\mathcal{M}_{\max})$ regardless of task complexity.
2. **Cold starts.** Models load on demand. Time-to-first-token (TTFT) is dominated by load time $t_{\text{load}} \gg t_{\text{infer}}$, making LLMs unsuitable as system-level services.
3. **Unverified confidence.** The output y has no external check. Self-consistency (sampling $y_1, \dots, y_k \sim \mathcal{M}_{\max}$) provides limited value: copies of the same model share blind spots.

1.2 The Thesis

An operating system does not load its process scheduler on demand. Cortex applies this principle to inference. Let $\mathcal{T} = \{\text{L0}, \text{L1}, \dots, \text{L7}\}$ be a set of tiers ordered by model capacity. We define a routing function $R : \mathcal{X} \rightarrow \mathcal{T}$ that maps each request to the *smallest sufficient tier*, and a confidence function $\mathcal{C} : \mathcal{Y} \rightarrow [0, 1]$ that determines whether escalation is needed.

2. Formal Framework

2.1 Tier Space

Definition 1 (Tier). A tier $\tau_i \in \mathcal{T}$ is a tuple (θ_i, v_i, h_i) where:

- $\theta_i \in \mathbb{R}^+$ is the parameter count (in billions)
- $v_i \in \mathbb{N}$ is the VRAM requirement (in MB)
- $h_i \in \{0, 1\}$ indicates whether the tier is always-resident

The tier ordering is $\tau_0 \prec \tau_1 \prec \dots \prec \tau_7$, where $\prec$ denotes strictly increasing capability and cost. The always-resident set is:

$$\mathcal{H} = \{\tau_i \in \mathcal{T} \mid h_i = 1\} = \{\text{L0}, \text{L1}, \text{L2}\} \quad (2)$$

Table 1: Tier specifications

Tier	OS Analogy	θ (B)	v (MB)	TTFT	Hot
L0	<code>init</code>	0.6	500	10ms	✓
L1	Syscall	1.7	1200	20ms	✓
L2	Shell	4	2800	40ms	✓
L3	Daemon	8	5000	60ms	
L4	Heavy daemon	14	9000	100ms	
L5	Workstation	32	18000	200ms	
L6	Full WS	72	42000	400ms	
L7	Network	∞	0	500ms	

2.2 Feasibility

Definition 2 (Feasible Tier). *Given a system with available VRAM budget V_{avail} , tier τ_i is feasible iff $v_i \leq V_{\text{avail}}$. The maximum feasible tier is:*

$$\tau_{\text{max}} = \max\{\tau_i \in \mathcal{T} \mid v_i \leq V_{\text{avail}} \wedge \tau_i \neq \text{L7}\} \quad (3)$$

The VRAM budget is computed as:

$$V_{\text{avail}} = \begin{cases} 0.75 \cdot V_{\text{unified}} & \text{Apple Silicon} \\ 0.80 \cdot V_{\text{discrete}} & \text{discrete GPU} \\ 0.75 \cdot V_{\text{RAM}} & \text{CPU-only} \end{cases} \quad (4)$$

The concurrent loading constraint ensures always-hot tiers fit alongside the working tier:

$$\sum_{\tau_i \in \mathcal{H}} v_i + v_k \leq V_{\text{avail}} \quad \text{for working tier } \tau_k \quad (5)$$

3. Routing

3.1 The Routing Function

Definition 3 (Router). *The routing function $R : \mathcal{X} \rightarrow \mathcal{T} \times [0, 1]$ maps a request x to a tier-confidence pair (τ, c) :*

$$R(x) = (\tau^*(x), \mathcal{C}_R(x)) \quad (6)$$

where $\tau^*(x)$ is the selected tier and $\mathcal{C}_R(x)$ is the routing confidence.

3.2 Complexity Estimation

The heuristic router estimates a complexity score $\kappa(x) \in [0, 1]$:

$$\kappa(x) = \min(1, \kappa_\ell(x) + \kappa_p(x) + \kappa_s(x) + \kappa_t(x)) \quad (7)$$

where:

$$\kappa_\ell(x) = \begin{cases} 0.0 & |x| < 50 \\ 0.1 & |x| < 200 \\ 0.2 & |x| < 500 \\ 0.3 & |x| < 2000 \\ 0.4 & \text{otherwise} \end{cases} \quad (8)$$

$$\kappa_p(x) = \sum_{p \in \mathcal{P}} w_p \cdot \mathbf{1}[p \text{ matches } x] \quad (9)$$

$$\kappa_s(x) = \min(0.2, 0.05 \cdot |\text{step indicators in } x|) \quad (10)$$

$$\kappa_t(x) = \min(0.15, 0.05 \cdot |\text{tool verbs in } x|) \quad (11)$$

The pattern weights w_p are: code patterns = 0.25, planning = 0.20, analysis = 0.15, tool use = 0.10.

3.3 Tier Selection

Let $\gamma : [0, 1] \rightarrow \mathcal{T}$ map complexity to tier and $\phi : \text{Category} \rightarrow \mathcal{T}$ map task category to minimum tier. The selected tier is:

$$\tau^*(x) = \min(\tau_{\text{max}}, \max(\gamma(\kappa(x)), \phi(\text{cat}(x)))) \quad (12)$$

Routing confidence decreases with the gap between ideal and actual tier:

$$\mathcal{C}_R(x) = \text{clip}(1.0 - 0.15 \cdot |\tau^* - \max(\gamma(\kappa), \phi(\text{cat}))|, 0.3, 1.0) \quad (13)$$

3.4 Model-Based Routing

When the L0 model $\mathcal{M}_0$ is loaded, routing uses the model itself:

$$R_{\mathcal{M}}(x) = \mathcal{M}_0(\text{"classify: " || } x_{[:500]}) \rightarrow (\tau, \text{cat}, c) \quad (14)$$

The L0 model outputs a structured JSON classification. If $\mathcal{M}_0$ fails, the system falls back to the heuristic R_H .

4. Cross-Family Confidence

4.1 Independence Argument

Let $\mathcal{F} = \{f_1, f_2, \dots, f_n\}$ be the set of model families (Qwen, Llama, Gemma, Granite, Phi, OLMo). Models within the same family share training data, architecture, and optimization — their failures are correlated. Models across families are approximately independent.

Proposition 1 (Cross-Family Confidence). *Let $\mathcal{M}_a \in f_i$ and $\mathcal{M}_b \in f_j$ with $f_i \neq f_j$. Let $p_a = P(\mathcal{M}_a \text{ correct})$ and $p_b = P(\mathcal{M}_b \text{ correct})$. If $\mathcal{M}_a$ and $\mathcal{M}_b$ agree on answer y , the posterior probability of correctness is:*

$$P(y \text{ correct} \mid \mathcal{M}_a = \mathcal{M}_b = y) = \frac{p_a p_b}{p_a p_b + (1 - p_a)(1 - p_b) \cdot \epsilon} \quad (15)$$

where ϵ is the probability of coincidental agreement on an incorrect answer. For independent models, $\epsilon \ll 1$, yielding $P \rightarrow 1$ as $\epsilon \rightarrow 0$.

Within the same family, ϵ is substantially higher due to correlated failure modes (shared training data, similar attention patterns). This is why cross-family agreement is a stronger signal.

4.2 Challenge Protocol

The challenge is triggered when $\mathcal{C}_R(x) < \alpha$, where $\alpha = 0.75$:

Algorithm 1 Cross-Family Challenge

Require: messages m , core response y_c from $\mathcal{M}_c \in f_{\text{core}}$, tier τ

- 1: $\mathcal{M}_d \leftarrow \text{SELECTCHALLENGER}(\tau, f_{\text{core}})$ $\triangleright$
different family
- 2: $y_d \leftarrow \mathcal{M}_d(m)$
- 3: $a \leftarrow \text{COMPAREANSWERS}(y_c, y_d)$
- 4: **if** $a \in \{\text{strong_agree}, \text{weak_agree}\}$ **then**
- 5: **return** y_c with boosted confidence
- 6: **else**
- 7: **return** $\text{ESCALATE}(\text{Swarm})$
- 8: **end if**

4.3 Answer Comparison

Two answers are compared using a composite similarity metric. Let σ_J denote Jaccard similarity over word sets:

$$\sigma_J(a, b) = \frac{|W(a) \cap W(b)|}{|W(a) \cup W(b)|} \quad (16)$$

where $W(\cdot)$ extracts the normalized word set. The comparison uses a weighted combination of conclusion similarity and full-text similarity:

$$\sigma(a, b) = 0.6 \cdot \sigma_J(C(a), C(b)) + 0.4 \cdot \sigma_J(a, b) \quad (17)$$

where $C(\cdot)$ extracts the first substantive sentence. Agreement is classified by thresholds:

Table 2: Agreement classification

Agreement Level	σ Range	Confidence
Strong agree	> 0.70	0.85
Weak agree	> 0.45	0.65
Ambiguous	> 0.25	0.45
Disagree	> 0.10	0.25
Strong disagree	≤ 0.10	0.10

5. Swarm Consensus

5.1 Fan-Out

When the challenger disagrees or initial confidence falls below $\beta = 0.50$, Cortex escalates to a swarm of n models from k families:

$$\mathcal{S} = \{(\mathcal{M}_1, f_1), (\mathcal{M}_2, f_2), \dots, (\mathcal{M}_n, f_n)\} \quad (18)$$

For *hard* problems: $n \in [3, 5]$, maximizing family diversity. For *hardest* problems: all available models.

5.2 Weight Function

Each model receives a vote weight based on its parameter count:

$$w(\theta) = \begin{cases} 1.0 & \theta \leq 1\text{B} \\ 1.5 & \theta \leq 4\text{B} \\ 2.0 & \theta \leq 8\text{B} \\ 2.5 & \theta \leq 14\text{B} \\ 3.0 & \theta \leq 32\text{B} \\ 4.0 & \theta \leq 70\text{B} \\ 5.0 & \text{otherwise} \end{cases} \quad (19)$$

5.3 Clustering

Responses are grouped into agreement clusters $\{G_1, G_2, \dots, G_m\}$ using greedy assignment. Response y_j is assigned to the first cluster G_i such that $\sigma(y_j, y_{G_i}^{\text{rep}}) > 0.45$ (weak agreement with the cluster representative). Otherwise, a new cluster is created.

5.4 Weighted Consensus

The winning cluster G^* maximizes total weight:

$$G^* = \arg \max_{G_i} \sum_{(\mathcal{M}_j, f_j) \in G_i} w(\theta_j) \quad (20)$$

The consensus answer is selected from the highest-weighted model in G^* :

$$y^* = y_{\arg \max_{j \in G^*} w(\theta_j)} \quad (21)$$

5.5 Swarm Confidence

Confidence is computed as:

$$\mathcal{C}_S = \text{clip}(\rho + \lambda_f - \lambda_s, 0, 1) \quad (22)$$

where:

$$\rho = \frac{\sum_{j \in G^*} w(\theta_j)}{\sum_{j=1}^n w(\theta_j)} \quad (\text{weighted agreement ratio}) \quad (23)$$

$$\lambda_f = \min(0.15, 0.05 \cdot |F(G^*)|) \quad (\text{family diversity bonus}) \quad (24)$$

$$\lambda_s = \max(0, 0.1 - 0.02 \cdot n) \quad (\text{small-swarm penalty}) \quad (25)$$

and $F(G^*)$ is the set of distinct families in the winning cluster.

Theorem 1 (Confidence Monotonicity). *For fixed agreement ratio ρ , the swarm confidence $\mathcal{C}_S$ is monotonically non-decreasing in both the number of agreeing families $|F(G^*)|$ and the total swarm size n . Additional independent agreement can only increase confidence.*

Proof. λ_f is non-decreasing in $|F(G^*)|$ (bounded by 0.15). λ_s is non-increasing in n (and becomes 0 for $n \geq 5$). Since $\mathcal{C}_S = \rho + \lambda_f - \lambda_s$, both terms move $\mathcal{C}_S$ upward. $\square$ $\square$

6. Escalation Pipeline

The full pipeline is a sequential decision process with three confidence thresholds:

$$\alpha = 0.75 \quad (\text{challenge threshold}) \quad (26)$$

$$\beta = 0.50 \quad (\text{swarm threshold}) \quad (27)$$

$$\delta = 0.30 \quad (\text{large swarm threshold}) \quad (28)$$

Algorithm 2 Cortex Process

Require: request x , messages m

```

1:  $(\tau, c) \leftarrow R(x)$  ▷ Route
2:  $y \leftarrow \mathcal{M}_\tau^{\text{core}}(m)$  ▷ Core model
3: if  $y = \text{null}$  then
4:    $(\tau, y) \leftarrow \text{ESCALATE}(\tau + 1 \dots \tau_{\text{max}})$ 
5: end if
6: if  $c < \alpha$  then ▷ Challenge
7:    $(a, c') \leftarrow \text{CHALLENGE}(m, y, \tau)$ 
8:   if  $a \notin \{\text{agree}\}$  then
9:     if  $c' < \delta$  then ▷ Large swarm
10:       $y, c'' \leftarrow \text{SWARM}(m, \tau, \text{LARGE})$ 
11:     else ▷ Small swarm
12:       $y, c'' \leftarrow \text{SWARM}(m, \tau, \text{SMALL})$ 
13:     end if
14:   end if
15: end if
16: return  $y$ 
```

The expected cost is substantially lower than always using $\mathcal{M}_{\text{max}}$. For a request distribution where fraction p_e of requests are “easy” (handled at L0–L2), p_m are “medium,” and p_h are “hard”:

$$\mathbb{E}[\text{cost}] = p_e \cdot c(\tau_{\text{low}}) + p_m \cdot 2c(\tau_{\text{mid}}) + p_h \cdot n \cdot c(\tau_{\text{mid}}) \quad (29)$$

Since empirically $p_e \gg p_h$, the expected cost is dominated by cheap, low-tier inference.

7. System Layer

7.1 Hardware Detection

On boot, Cortex constructs a system profile $\mathcal{P}$:

$$\mathcal{P} = (\text{CPU}, \text{GPU}, \text{RAM}, \text{Backends}, \text{OS}) \quad (30)$$

Detection covers NVIDIA (via `nvidia-smi`), AMD (via `rocm-smi`), Apple Silicon (via `sysctl`), and CPU features (AVX2, AVX-512, NEON, AMX). The feasible tier set $\mathcal{T}_{\mathcal{P}} \subseteq \mathcal{T}$ is computed at boot time and cached.

7.2 Model Manager

The Model Manager implements the `systemd` analogy:

- **Boot:** loads $\mathcal{H} = \{\text{L0}, \text{L1}, \text{L2}\}$
- **Load:** instantiates on-demand tiers when first requested
- **Evict:** LRU eviction under VRAM pressure
- **Health:** periodic liveness checks, automatic restart

The VRAM scheduler solves a bin-packing problem: maximize the number of concurrently loaded tiers subject to $\sum v_i \leq V_{\text{avail}}$, with always-hot tiers having priority.

7.3 Persistent Memory

All state persists in SQLite (`~/cortex/cortex.db`) with WAL mode:

- **Threads:** conversation state and message history
- **Audit log:** every routing decision, tier, model, confidence, TTFT, and escalation path (append-only)
- **KV cache index:** prefix hashes for cache reuse across requests
- **Policies:** hierarchical config (`thread > app > global`)
- **Usage stats:** daily aggregates by tier and model

7.4 API Compatibility

The daemon accepts three formats and normalizes to a common internal representation:

Request_{any} $\xrightarrow{\text{normalize}}$ ChatCompletions $\xrightarrow{\text{Cortex}}$ Response $\xrightarrow{\text{forma}}$

Table 3: API format support

Format	Status
OpenAI Chat Completions	Native
OpenAI Responses API	Translated
Anthropic Messages API	Translated
Tool calling	Pass-through
Multimodal	Text-extracted

8. Performance

8.1 TTFT Model

Time-to-first-token for a request routed to tier τ_i with resident status h_i :

$$\text{TTFT}(\tau_i) = \begin{cases} t_{\text{infer}}(\tau_i) & h_i = 1 \text{ (always-hot)} \\ t_{\text{load}}(\tau_i) + t_{\text{infer}}(\tau_i) & h_i = 0 \text{ (cold)} \\ t_{\text{infer}}(\tau_i) & h_i = 0 \text{ (warm, cached)} \end{cases} \quad (31)$$

Since $\mathcal{H}$ handles the majority of requests (routing, classification, tool calls, drafts), the expected TTFT across all requests is:

$$\mathbb{E}[\text{TTFT}] = \sum_i P(\tau_i) \cdot \text{TTFT}(\tau_i) \approx P(\mathcal{H}) \cdot \bar{t}_{\mathcal{H}} \quad (32)$$

where $\bar{t}_{\mathcal{H}} \leq 40\text{ms}$.

8.2 Platform Performance

Table 4: Expected TTFT by platform (L2 tier)

Platform	Backend	TTFT
Linux + H100	vLLM (AWQ)	20–50ms
Linux + RTX 4090	llama.cpp	30–80ms
macOS + M4 Ultra	llama.cpp (Metal)	50–150ms
Linux + RX 7900	llama.cpp (ROCm)	40–100ms
CPU-only	llama.cpp (AVX2)	200–2000ms

8.3 Graceful Degradation

Let V be the available VRAM. Cortex’s capability degrades smoothly:

$$\text{capability}(V) = \begin{cases} \text{Full stack (L0–L6)} & V \geq 42\text{GB} \\ \text{Strong local (L0–L5)} & V \geq 18\text{GB} \\ \text{Standard (L0–L4)} & V \geq 9\text{GB} \\ \text{Compact (L0–L3)} & V \geq 5\text{GB} \\ \text{Minimal (L0–L2)} & V \geq 2.8\text{GB} \\ \text{Reflex only (L0)} & V \geq 0.5\text{GB} \end{cases} \quad (33)$$

At every level, L7 (remote frontier) remains available as a network fallback. The system always functions — analogous to booting from a minimal `initramfs`.

9. GGUF-First Format Strategy

Cortex standardizes on GGUF as the universal model file format. A single quantized file $\mathcal{M}_{\text{gguf}}$ executes identically across:

$$\mathcal{M}_{\text{gguf}} \rightarrow \{\text{CUDA, Metal, ROCm, CPU}\} \quad (34)$$

This is a deliberate architectural choice. Let n_p be the number of platforms and n_f the number of format variants. Without GGUF, the model catalog grows as $O(n_p \cdot n_f)$. With GGUF:

$$|\text{catalog}| = |\mathcal{T}| \cdot (1 + |\mathcal{F}_{\text{challenge}}|) \quad (35)$$

For NVIDIA users running vLLM, AWQ-quantized variants are optionally available for improved prefix caching, but GGUF remains the universal fallback.

10. Design Principles

1. **Start small.** $\tau^*(x) = \min\{\tau \in \mathcal{T} \mid \tau \text{ sufficient for } x\}$.
2. **Measure confidence.** Cross-family verification provides an empirical signal that single-model inference lacks.
3. **One format everywhere.** GGUF eliminates platform-specific model management.
4. **Always available.** $\mathcal{H}$ is loaded at boot and never evicted.
5. **Degrade gracefully.** If hardware supports only $\mathcal{H}$, the system still functions. L7 is the network fallback.
6. **Audit everything.** Every routing decision, escalation, and latency is logged for continuous improvement.

11. Future Work

- **LLM-as-judge aggregation:** replace Jaccard-based σ with a small judge model for nuanced agreement detection
- **Persistent credibility:** track per-model accuracy over time and adjust $w(\theta)$ dynamically
- **Adaptive routing:** fine-tune $\mathcal{M}_0$ on audit log data to improve classification accuracy
- **Multi-GPU scheduling:** VRAM-aware placement across multiple devices

- **Speculative decoding:** use L0 as a draft model for higher tiers
- **Embedding memory:** vector search over conversation history for long-term recall

12. Conclusion

Cortex reframes LLM inference as an operating system problem. The L0 router acts as PID 1, the cross-family challenge provides empirical confidence measurement, and the swarm offers weighted consensus for the hardest problems. By routing to the smallest capable model, the system is faster (lower TTFT), more reliable (measured confidence via independent verification), and universally deployable (GGUF-first).

The question is not whether AI should be integrated into the operating system. It is whether the operating system should be redesigned around AI. Cortex is an argument that it should.

Cortex is open source under the MIT License.
<https://github.com/elevate-foundry>